The Journal of Open Source Software

FLIMKit: a Python toolkit for reconvolution fitting, lifetime-distribution fitting, and phasor analysis of FLIM data

Alexander Hunt [1¶], **Ava Russell** [1], **Layla Mathieson** [1], **Zhen Yuan Yeo** [2], **Matthieu Veremen** [1], **and Ahsan R. Akram** [1]

1 Centre for Inflammation Research, Institute for Regeneration and Repair, The University of Edinburgh, Edinburgh, United Kingdom **2** Department of Statistics and Data Science, National University of Singapore, Singapore ¶ Corresponding author

DOI: 10.xxxxxx/draft

Software

- Review
- Repository
- Archive

Editor:

Submitted: 03 September 2026
Published: unpublished

License
Authors of papers retain copyright and release the work under a Creative Commons Attribution 4.0 International License (CC BY 4.0).

DRAFT

Summary

Fluorescence lifetime imaging microscopy (FLIM) measures the time a fluorescent molecule remains in its excited state before emitting a photon. This lifetime is determined by the local chemical environment of the molecule (pH, oxygen tension, ion binding, or proximity to an acceptor through Förster resonance energy transfer, FRET) rather than by its concentration or by the intensity of the excitation light, which makes it well suited to quantitative, excitation-independent measurement (Datta et al., 2020). The molecule may be an introduced label or an endogenous species such as the metabolic cofactors NAD(P)H and FAD, so FLIM is applicable label-free to tissue autofluorescence as well as to stained preparations. This sensitivity to the local environment underpins a growing body of translational work: autofluorescence lifetime alone classifies human T cells by activation state with 97–99% accuracy, and by activation state and subtype together with 97% accuracy, without any exogenous label or destructive assay (Walsh et al., 2021). Label-free lifetime imaging has also been used to discriminate lung cancer from adjacent non-cancerous tissue during fibre-based imaging (Fernandes et al., 2024), to generate virtual haematoxylin-and-eosin histology from unstained tissue (Wang et al., 2024), and to predict EGFR mutation status in lung adenocarcinoma without staining or sequencing (Zang et al., 2026). Realising that potential at scale depends on analysis software that is reproducible, scriptable, and not tied to a single instrument vendor.

FLIMKit is a Python toolkit for the analysis of FLIM data acquired on time-correlated single-photon counting (TCSPC) instruments (Shcheslavskiy et al., 2025). PicoQuant `.ptu`, Becker & Hickl `.sdt`, Photonscore `.photons`, and, experimentally, ISS time-tag files are read through one shared file abstraction, and three analysis routes are supported: reconvolution fitting of discrete multi-exponential models, fitting of continuous lifetime distributions, and calibrated phasor analysis (Digman et al., 2008; Ranjit et al., 2018). Per-pixel fitting is GPU-accelerated on Apple Silicon, NVIDIA CUDA, and AMD ROCm hardware. FLIMKit may be operated as a desktop application, a guided terminal interface, standalone command-line scripts, or a Python API, and exports results as OME-TIFF, GeoJSON for QuPath, and CSV. A plugin registry allows analysis tools, file formats, and phasor filters to be added without modifying the package.

Statement of need

Laboratories operating Leica FALCON/STELLARIS, Becker & Hickl SPC, Photonscore LINCam, or ISS FastFLIM/Vista instruments typically analyse their FLIM data using the software bundled by the instrument vendor, such as LAS X, SPCImage, or VistaVision. Given that these

packages are typically closed-source, graphical-only, and built around the file format of their own instrument, batching and scripting are difficult, and analyses beyond those the vendor anticipated – whether more detailed, or drawing on newer methods – are correspondingly hard to implement. A facility operating more than one TCSPC platform is consequently left without a single reproducible pipeline spanning its instruments. FLIMKit addresses this through a single `FLIMFile` abstraction, to which every supported format is decoded, such that the same fitting, phasor, stitching, and export code operates regardless of the instrument of origin.

Other open-source software addresses a separate part of this problem. FLIMfit (Warren et al., 2013) provides well-established reconvolution and global fitting; its phasor functionality is limited to phasor-based segmentation rather than a general phasor-analysis workflow. It is built on MATLAB and reads several vendor formats through its FlimReader library. Its GitHub repository carries no release newer than 5.0.3 (June 2017), although the project website distributes version 5.1.1 (July 2018). On the phasor side, PhasorPy (Gohlke et al., 2026), the `napari-flim-phasor-plotter` and `napari-phasors` plugins (Pannunzio et al., 2026; Zoccoler et al., 2024), FLIMPA (Kapsiani et al., 2025), and GSLab (Vallmitjana et al., 2025) all implement the phasor method described by Digman et al. (Digman et al., 2008) and formalised as a fit-free protocol by Ranjit et al. (Ranjit et al., 2018). FLIMKit does not reimplement that method: PhasorPy is adopted as its phasor backbone, and calibration, interactive cursors, spatial filtering, and ROI-based batch export are layered upon it. FLIMPA invites close comparison, being an actively maintained, standalone desktop application with its own region-of-interest (ROI) selection tool. None of these packages, however, perform iterative reconvolution fitting against a measured instrument response function (IRF). GSLab is additionally written as a MATLAB app, although a compiled Windows installer is available for users without a MATLAB licence and installs the required MATLAB Runtime.

The field divides instead into fitting tools and phasor tools, the latter of which do not fit decays. Among the fitting tools, FLIMLib (FLIMLib contributors, 2026) remains actively maintained, but is provided as a C library used by the Windows-only TRI2 application and by the FLIMJ plugin for ImageJ/Fiji; neither it nor FLIMfit provides a cross-platform application pairing fitting with a calibrated phasor workflow. Among the packages compared here, FLIMKit is therefore the only open-source tool combining iterative reconvolution against a measured IRF with a calibrated phasor workflow in a scriptable, GPU-accelerated package. FLIMKit is intended for laboratories and imaging facilities requiring vendor-independent FLIM analysis spanning these routes within one tool, and is aimed at both ends of the user range: those who wish to open an acquisition, fit it, and export the result through a graphical interface without writing code, and those constructing automated pipelines that require a command-line or programmatic interface. PicoQuant `.ptu`, Becker & Hickl `.sdt`, and Photonscore `.photons` support have been validated against real instrument files; ISS support is derived from the vendor's format specifications and has not yet been verified against real ISS acquisitions, and is therefore offered as experimental.

State of the field

Table 1: Comparison of FLIM analysis software along axes relevant to FLIMKit's design. FLIMKit's phasor analysis is implemented on PhasorPy rather than independently of it.

Software	Access	Formats	Fitting / phasor	Scriptable
LAS X / SPCImage / VistaVision	Proprietary	Vendor-locked	Yes / partial	No
FLIMfit (Warren et al., 2013)	Open source	Several (via FlimReader)	Yes / partial	Partial

DRAFT

Software	Access	Formats	Fitting / phasor	Scriptable
PhasorPy (Gohlke et al., 2026)	Open source	Many (readers)	No / yes	Yes
napari-flim-phasor-plotter / napari-phasors (Pannunzio et al., 2026; Zoccoler et al., 2024)	Open source	Several	No / yes	Via napari
FLIMPA (Kapsiani et al., 2025)	Open source	PTU, .sdt, .tif	No / yes (+ROI)	No
GSLab (Vallmitjana et al., 2025)	Open source	Several	No / yes	Via MATLAB
FLIMKit	Open source	PTU, .sdt, Photonscore, ISS (exp.)	Yes / yes	Yes

Table 1 summarises how FLIMKit relates to the packages described above. To our knowledge, FLIMKit is the only package that is open-source, supports multiple instrument vendors behind one abstraction, performs both reconvolution fitting against a measured IRF and calibrated phasor analysis, and is scriptable with GPU acceleration built in. FLIMfit and the phasor-only tools each address part of this scope. The phasor-focused packages remain more mature in that domain, and FLIMKit builds directly upon PhasorPy rather than competing with it. The contribution of FLIMKit is therefore integrative: a single file abstraction, both analysis families within one interface, and three validated instrument vendors alongside an experimental fourth, in place of assembling several single-purpose tools manually. The desktop application has been designed so that a complete analysis – loading an acquisition, fitting it, defining regions of interest, and exporting results – may be carried out entirely through the graphical interface, with little or no programming knowledge required.

Software design

The principal design decision in FLIMKit is the `FLIMFile` format abstraction. To avoid duplicating work that is already well established, instrument formats are not reimplemented where mature decoders exist: each reader wraps an existing library, `ptufile` and `sdtfile` (both by Gohlke) for PicoQuant and Becker & Hickl data and `photonsfile` (Hunt & Akram, 2026) for Photonscore, and reduces its output to a common `(Y, X, H)` decay-cube-and-metadata contract, so that the fitting, phasor, stitching, and ROI code need be written only once and remains format-agnostic. Photonscore `.photons` files reach that cube by a different route: rather than arriving pre-binned as scan-line images, each photon carries its own (x, y) position and TCSPC micro-time (the arrival time of the photon relative to the excitation pulse), so the image is formed by binning positions onto a pixel grid and the decay by histogramming the micro-time. That reader, `photonsfile`, was written for FLIMKit and released as a standalone package, validated bit-exact against the vendor SDK. The abstraction does constrain what a new reader may expose, since no instrument-specific field is passed further into the pipeline, but it yields one tested code path rather than one per format.

Per-pixel fitting is the principal computational cost in FLIMKit: a 1024×1024 acquisition requires of the order of a million independent non-linear fits, one for each pixel. This workload

DRAFT

suits a GPU well, but there is no single GPU programming interface common to the Apple Silicon, NVIDIA, and AMD hardware found across different laboratories. A GPU backend abstraction (MLX, PyTorch MPS/CUDA/ROCm, or a CPU fallback, detected automatically at install time) therefore allows the same `fit_per_pixel()` call to execute on whichever hardware is available, without separate fitting code for each backend. The single global ('summed') fit is deliberately restricted to the CPU, since it does not constitute a bottleneck and does not warrant the additional complexity.

Two lifetime models are provided for two different situations: a discrete multi-exponential model, which assumes a fixed and small number of decaying species and may be set to one, two, or three components, and a continuous Gaussian/Lorentzian $\alpha(\tau)$ distribution model, following Lakowicz (Lakowicz, 2006), fitted as either one (unimodal) or two (bimodal) distributions, for genuinely heterogeneous lifetime populations that a fixed-species model would misrepresent. Three is the upper limit on the discrete model because each further component adds two free parameters, and the recovered lifetimes cease to be separable at realistic per-pixel photon counts. The distribution model is of relevance to autofluorescence, where the measured decay arises from a distribution of endogenous species rather than a small number of discrete ones. A separate time-varying background correction, following the method described by Görlitz et al. (Görlitz et al., 2017), subtracts a measured and scaled time-varying background rather than a flat offset alone.

Multi-tile acquisitions are stitched into a single mosaic prior to fitting. Tile positions are read from the stage metadata of the acquisition and refined by cross-correlation registration between neighbouring tiles, following the approach of Preibisch et al. (Preibisch et al., 2009), so that per-pixel fitting and phasor analysis operate on one continuous field of view rather than tile by tile. For batch ROI workflows, per-cell regions may be obtained by either of two routes. Segmentation with Cellpose-SAM (Pachitariu et al., 2025) is part of FLIMKit itself, installed through an optional dependency group and included in the compiled desktop builds: cell masks are generated directly from the summed intensity image, so that per-cell lifetime and phasor statistics may be exported without a separate segmentation step in another package (Figure 1). Alternatively, masks segmented elsewhere may be imported, either as GeoJSON regions written by any external tool, or live through the QuPath and Fiji bridge add-ons described below, which exchange images and regions with a running FLIMKit over a local interface.

DRAFT

The Journal of Open Source Software

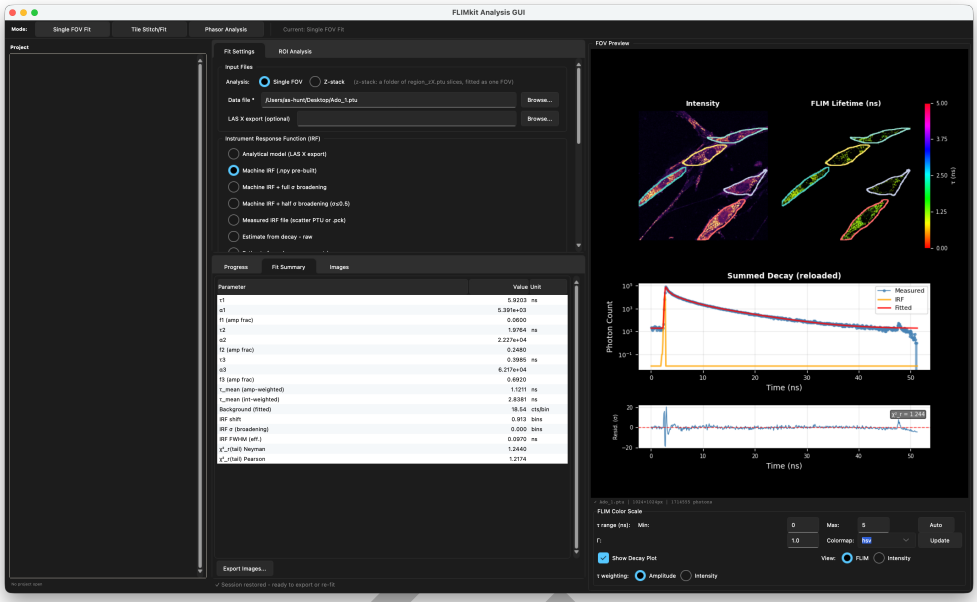


Figure 1: The FLIMKit desktop interface in Single FOV Fit mode. Five freehand regions of interest have been drawn over individual cells and are outlined on both images in the FOV Preview panel, which shows the intensity image and the corresponding per-pixel lifetime map above the summed decay with its reconvolution fit and IRF, and the weighted residuals. The Fit Summary panel lists the recovered tri-exponential parameters (τ_1–τ_3 with their amplitude fractions), the amplitude- and intensity-weighted mean lifetimes, the fitted background and IRF shift, and reduced χ^2 (a goodness-of-fit statistic normalised by the degrees of freedom, with values near unity indicating an adequate fit). Per-ROI mean, median, and standard deviation of the fitted lifetime, alongside photon counts, are reported in the Regions table under the ROI Analysis tab. Data are a PicoQuant `.ptu` acquisition (1024×1024 pixels, 1.7×10^6 photons).

Capability that a particular laboratory requires, but that does not belong in a general FLIM toolkit, is added through a plugin registry rather than by modifying FLIMKit itself. A plugin is an ordinary Python module that registers against one of six hooks, covering analysis tools, ROI panel buttons, startup actions, file formats, format sniffers, and phasor filters, and that declares the registry API version it was written against; a declared version that does not match is refused rather than partially loaded. FLIMKit's own tools are registered through the same registry, so the extension path is exercised by ordinary use and not solely by third-party code. Plugins are discovered in a user directory, in directories named in the configuration or in an environment variable, and in installed packages declaring a `flimkit.plugins` entry point. A plugin raising an exception during import has its registrations withdrawn and cannot prevent the application from starting, and the mechanism as a whole may be disabled to obtain a reproducible baseline. The hook signatures are held fixed at version 1 and a fixture plugin written against them is loaded by the test suite on every run, so that a change breaking the contract fails in continuous integration rather than in a user's installation. Three add-ons are distributed as separate packages on this basis rather than being built into FLIMKit: the bridges to QuPath and to Fiji referred to above, and a time-resolved anisotropy package, which adds a fitting model the core does not provide.

Serving both audiences from one codebase is the reason for the four entry points. The desktop application, terminal interface, command-line scripts, and Python API all call the same underlying fitting and phasor functions, so an analysis performed interactively can be reproduced in a script without alteration, and the graphical and scripted implementations cannot diverge over time. Correctness is assessed using a test suite of 42 test modules (`flimkit_tests/`)

DRAFT

covering CPU and GPU numerical parity, ground-truth lifetime recovery, and format-specific decoding, together with an installation validation script (`validate_installation.py`). A GitHub Actions workflow builds self-contained desktop applications for macOS, Linux, and Windows on each tagged release, and publishes them as release assets alongside the source; these compiled builds require neither an existing Python installation nor any use of the command line, so that FLIMKit may be installed by users who do not maintain a Python environment. A Docker image is also provided, which runs the full desktop interface headlessly on a virtual X server and serves it to a browser over noVNC, with a documented TrueNAS SCALE deployment path, chosen so that FLIMKit may be deployed on shared imaging-facility hardware rather than individual workstations alone.

Research impact statement

FLIMKit has been developed and applied to the analysis of PicoQuant and Leica FALCON/STELLARIS acquisitions within an ongoing research program at the Centre for Inflammation Research, University of Edinburgh, where it is the routine analysis route for FLIM data and for work currently in preparation.

Format support has been developed in direct collaboration with the instrument vendors. Becker & Hickl format support was constructed and verified against sample data and file-format documentation provided by the vendor; Photonscore format support was validated against a sample `.photons` file and SDK documentation provided by Photonscore, and the resulting reader released as a standalone package (Hunt & Akram, 2026); ISS format specifications were likewise provided by ISS following a discussion on the CONFOCALMICROSCOPY listserv. FLIMKit is listed among available phasor-analysis tools in the PhasorPy documentation. Beyond the authors' group, it has been applied to independently acquired datasets at the CRUK Scotland Institute and the National University of Singapore. This represents early-stage, exploratory use rather than validated deployment, and broader external adoption remains a priority for the project.

AI usage disclosure

Anthropic's Claude was used as an assistant for parts of the graphical user interface implementation, the add-on system, compiled-application packaging, Docker and TrueNAS deployment scripting, and code debugging, under the direction of the lead author (A.H.). Contributions by Z.Y.Y. were developed with assistance from OpenAI's GPT-5.6 Sol, operated through Hermes Agent by Nous Research, supporting implementation, testing, and documentation; Z.Y.Y. directed and reviewed that work and remains responsible for the submitted implementations. All scientific design, comprising the fitting models, IRF handling, phasor methods, validation approach, and overall software architecture, is the authors' own work. AI-assisted code was reviewed and tested by the authors, including through the automated test suite, the installation validation script, and CPU/GPU numerical parity tests, before being merged. No AI tool was used to generate or to verify the scientific or statistical methods described in this paper.

Acknowledgements

We thank Christoph Gohlke for PhasorPy, `ptufile`, `lfdfiles`, and `sdtfile`, upon which FLIMKit depends. Dr. Jens Balke and Enzo Marscheck (Becker & Hickl) for `.sdt` format documentation and sample data; Photonscore GmbH for the LINCam SDK, a sample `.photons` file, and for open-sourcing the D7 format; and Jeff Liao (ISS) for ISS time-tag format specifications.

DRAFT

References

Datta, R., Heaster, T. M., Sharick, J. T., Gillette, A. A., & Skala, M. C. (2020). Fluorescence lifetime imaging microscopy: Fundamentals and advances in instrumentation, analysis, and applications. *Journal of Biomedical Optics*, *25*(7), 071203. https://doi.org/10.1117/1.JBO.25.7.071203

Digman, M. A., Caiolfa, V. R., Zamai, M., & Gratton, E. (2008). The phasor approach to fluorescence lifetime imaging analysis. *Biophysical Journal*, *94*(2), L14–L16. https://doi.org/10.1529/biophysj.107.120154

Fernandes, S., Williams, E., Finlayson, N., Stewart, H., Dhaliwal, C., Dorward, D. A., Wallace, W. A., Akram, A. R., Stone, J., Dhaliwal, K., & Williams, G. O. S. (2024). Fibre-based fluorescence-lifetime imaging microscopy: A real-time biopsy guidance tool for suspected lung cancer. *Translational Lung Cancer Research*, *13*(2), 355–361. https://doi.org/10.21037/tlcr-23-638

FLIMLib contributors. (2026). *FLIMLib: A package for exponential curve fitting of fluorescence lifetime image data*. https://flimlib.github.io/.

Gohlke, C., Pannunzio, B., Schuty, B., Donato, M., & Blanco, R. (2026). *PhasorPy: An open-source python library for phasor analysis of fluorescence lifetime and hyperspectral images* (Version 0.12). Zenodo. https://doi.org/10.5281/zenodo.21461922

Görlitz, F., Kelly, D. J., Warren, S. C., Alibhai, D., West, L., Kumar, S., Alexandrov, Y., Munro, I., Garcia, E., McGinty, J., Talbot, C., Serwa, R. A., Thinon, E., Paola, V. da, Murray, E. J., Stuhmeier, F., Neil, M. A. A., Tate, E. W., Dunsby, C., & French, P. M. W. (2017). Open source high content analysis utilizing automated fluorescence lifetime imaging microscopy. *Journal of Visualized Experiments*, *119*, e55119. https://doi.org/10.3791/55119

Hunt, A., & Akram, A. (2026). *Photonsfile: A pure-python reader for photonscore LINCam .photons (D7) files*. Zenodo. https://doi.org/10.5281/zenodo.21360199

Kapsiani, S., Läubli, N. F., Ward, E. N., Shehata, M., Kaminski, C. F., & Kaminski Schierle, G. S. (2025). FLIMPA: A versatile software for fluorescence lifetime imaging microscopy phasor analysis. *Analytical Chemistry*. https://doi.org/10.1021/acs.analchem.5c00495

Lakowicz, J. R. (2006). *Principles of fluorescence spectroscopy* (3rd ed.). Springer. https://doi.org/10.1007/978-0-387-46312-4

Pachitariu, M., Rariden, M., & Stringer, C. (2025). Cellpose-SAM: Superhuman generalization for cellular segmentation. *bioRxiv*. https://doi.org/10.1101/2025.04.28.651001

Pannunzio, B., Zoccoler, M., Rivero, J., Schuty, B., & Gohlke, C. (2026). *Napari-phasors* (Version 0.7.2). Zenodo. https://doi.org/10.5281/zenodo.22234106

Preibisch, S., Saalfeld, S., & Tomancak, P. (2009). Globally optimal stitching of tiled 3D microscopic image acquisitions. *Bioinformatics*, *25*(11), 1463–1465. https://doi.org/10.1093/bioinformatics/btp184

Ranjit, S., Malacrida, L., Jameson, D. M., & Gratton, E. (2018). Fit-free analysis of fluorescence lifetime imaging data using the phasor approach. *Nature Protocols*, *13*(9), 1979–2004. https://doi.org/10.1038/s41596-018-0026-5

Shcheslavskiy, V. I., Shirmanova, M. V., Yashin, K. S., Rück, A. C., Skala, M. C., & Becker, W. (2025). Fluorescence lifetime imaging techniques—a review on principles, applications and clinical relevance. *Journal of Biophotonics*, *18*(12), e202400450. https://doi.org/10.1002/jbio.202400450

Vallmitjana, A., Torrado, B., Durkin, A. F., Dvornikov, A., Rajil, N., Ranjit, S., & Balu, M. (2025). GSLab: Open-source platform for advanced phasor analysis in fluorescence

DRAFT

microscopy. *Bioinformatics*, *41*(4), btaf162. https://doi.org/10.1093/bioinformatics/btaf162

Walsh, A. J., Mueller, K. P., Tweed, K., Jones, I., Walsh, C. M., Piscopo, N. J., Niemi, N. M., Pagliarini, D. J., Saha, K., & Skala, M. C. (2021). Classification of t-cell activation via autofluorescence lifetime imaging. *Nature Biomedical Engineering*, *5*(1), 77–88. https://doi.org/10.1038/s41551-020-0592-z

Wang, Q., Akram, A. R., Dorward, D. A., Talas, S., Monks, B., Thum, C., Hopgood, J. R., Javidi, M., & Vallejo, M. (2024). Deep learning-based virtual H&E staining from label-free autofluorescence lifetime images. *Npj Imaging*, *2*(1), 17. https://doi.org/10.1038/s44303-024-00021-7

Warren, S. C., Margineanu, A., Alibhai, D., Kelly, D. J., Talbot, C., Alexandrov, Y., Munro, I., Katan, M., Dunsby, C., & French, P. M. W. (2013). Rapid global fitting of large fluorescence lifetime imaging microscopy datasets. *PLOS ONE*, *8*(8), e70687. https://doi.org/10.1371/journal.pone.0070687

Zang, Z., Dorward, D. A., Ihuoma, S., Akram, A. R., & Wang, Q. (2026). Label-free prediction of EGFR mutation status using fluorescence lifetime imaging and deep learning in lung adenocarcinoma. *Cancer Research*. https://doi.org/10.1158/0008-5472.CAN-25-5589

Zoccoler, M., Iarovenko, S., & Wetzker, C. (2024). *Napari-flim-phasor-plotter* (Version 0.1.0). Zenodo. https://doi.org/10.5281/zenodo.12620956

DRAFT